Retrieval-Augmented Generation: System Architecture & Ingestion

1.1 Document Ingestion and Chunking Strategy

Retrieval-Augmented Generation (RAG) bridges external domain knowledge with neural generative models. The document ingestion subsystem extracts text from PDF documents page by page while preserving metadata. Text chunking is configured to a chunk size of 800 characters with an overlap of 100 characters. The overlap guarantees that critical semantic context spanning sentence boundaries is not split across separate fragments. A RecursiveCharacterTextSplitter operates hierarchically, evaluating double newlines, single newlines, and spaces. Each chunk is tagged with its origin filename and 1-based page index to enable verifiable document provenance.

1.2 Dense Vector Embeddings

Extracted text chunks are mapped into high-dimensional semantic vector spaces using neural embedding models. Standard embedding models generate 1536-dimensional dense vectors where semantic proximity corresponds to vector cosine similarity. Normalized vector embeddings allow rapid computation of similarity metrics during the retrieval phase.

Vector Databases and Similarity Search with FAISS

2.1 FAISS Indexing Mechanics

Vector similarity search in this system is driven by FAISS (Facebook AI Similarity Search). For deterministic and exact nearest-neighbor retrieval, FAISS employs an IndexFlatIP (Inner Product) structure. Because all chunk embeddings are L2-normalized upon creation, the inner product is mathematically identical to cosine similarity. During query execution, the vector store retrieves the top-k most relevant chunks, where k defaults to 4. Each retrieved chunk provides its similarity score, allowing the application to measure context distance.

2.2 Persistence and Collection Identity

FAISS indexes and their associated document stores are serialized to disk to support persistent restarts. A unique collection identifier is computed from the hash of indexed document names and chunk counts. This collection ID prevents cache contamination when new documents are added or existing collections are re-indexed.

Semantic Caching Principles, TTL, and Invalidation

3.1 Semantic Similarity Thresholds

Semantic caching prevents redundant vector searches and LLM inferences by storing previously asked queries and answers. Unlike exact key-value caches that require identical query strings, semantic caching compares query embedding vectors. When an incoming query exhibits a cosine similarity score greater than or equal to the threshold of 0.90 against a cached query, a cache hit is triggered. The cached answer and chunk citations are returned immediately. A cache hit reduces latency to under 20 milliseconds and completely eliminates LLM token consumption, saving 100% of LLM cost.

3.2 Cache Invalidation and Time-To-Live

To ensure stale information is never returned, cache entries enforce a configurable Time-To-Live (TTL) of 86400 seconds (24 hours). Expired entries are automatically purged during lookup. Furthermore, modifying the underlying document index immediately invalidates the collection cache, ensuring strict consistency.

Cost-Aware LLM Routing and Heuristic Classification

4.1 Dual-Tier Routing Architecture

The system deploys a two-tier model architecture to optimize inference economics: Tier 1 (SIMPLE) uses gpt-3.5-turbo for direct factual queries; Tier 2 (COMPLEX) uses gpt-4o-mini for synthesis, multi-step analysis, and comparative reasoning. A zero-cost heuristic router analyzes query complexity across four key signals: keyword density (analyzing words such as compare, analyze, evaluate, summarize, synthesize), query word count, multi-part sentence structures, and the number of retrieved context chunks.

4.2 Decision Threshold and Economic Impact

The router computes a normalized complexity score between 0.0 and 1.0. Queries with complexity scores below 0.45 are routed to the SIMPLE tier, while scores of 0.45 or greater route to the COMPLEX tier. By routing straightforward questions to the lower-cost model while reserving the advanced model for demanding prompts, the architecture achieves an estimated 40% to 60% cost reduction without degrading answer quality on complex tasks.

Production Observability, Telemetry, and Error Handling

5.1 Telemetry and Traceability

Every query generates an end-to-end trace with a unique UUID request identifier. The trace records stage latencies (cache lookup, retrieval, LLM call, and total duration), token usage, estimated dollar cost, and provenance citations. Percentile metrics (p50 and p95 latency) are tracked to monitor tail latency. Sensitive information, including API tokens, is strictly redacted prior to logging or telemetry serialization.

5.2 Fault Tolerance and Resiliency

The production RAG pipeline incorporates defensive error handling across all failure modes: Empty or whitespace queries return clear validation messages without invoking downstream services. Corrupted, empty, or unreadable PDFs are caught during ingestion with descriptive HTTP 422 errors. When an LLM API call fails due to rate limits or network timeouts, the system returns a graceful error response. If a question cannot be answered from the provided documents, the system adheres to strict grounding and replies: 'I don't have enough information in the provided documents to answer that.'